Boolean Algebra

Contemporary Digital Electronic Systems — Chapter 5

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. Multivariable Theorems
3. De Morgan's Theorems
4. Expressing Logic Circuits
5. Canonical Forms
6. Don't Cares
7. Algebraic Simplification
8. Summary

Why This Chapter Matters

A direct translation of a truth table into gates almost always wastes gates. Boolean algebra lets you *simplify* an expression and recover a smaller, faster circuit with the same truth table. It is also the vocabulary of every later design procedure.

After this chapter, you can:

1. State and apply the laws of Boolean algebra.
2. Convert between truth tables, expressions and circuits.
3. Write a function in canonical SOP and POS form.
4. Simplify an expression algebraically.
5. Apply De Morgan's theorems to complement and rewrite expressions.

© Muním Zabidi

Why are two circuits with the same truth table worth distinguishing?

What does “better” mean when comparing them? We answer this at the end of the chapter.

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 Multivariable Theorems

3 De Morgan's Theorems

4 Expressing Logic Circuits

5 Canonical Forms

6 Don't Cares

7 Algebraic Simplification

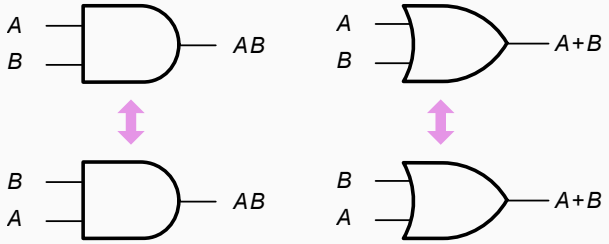
8 Summary

Boolean Theorems T1–T14

	Theorem	AND form	OR form
T1	Annulment	$A \cdot 0 = 0$	$A + 1 = 1$
T2	Identity	$A \cdot 1 = A$	$A + 0 = A$
T3	Idempotency	$AA = A$	$A + A = A$
T4	Complements	$A\overline{A} = 0$	$A + \overline{A} = 1$
T5	Involution		$\overline{\overline{A}} = A$
T6	Commutative	$AB = BA$	$A + B = B + A$
T7	Associative	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
T8	Distributive	$A(B + C) = AB + AC$	$A + BC = (A + B)(A + C)$
T9	Uniting	$AB + A\overline{B} = A$	$(A + B)(A + \overline{B}) = A$
T10	Absorption	$A + AB = A$	$A(A + B) = A$
T11	Redundancy	$A + \overline{A}B = A + B$	$(A + \overline{B})B = AB$
T12	Consensus	$AB + \overline{A}C = AB + BC + \overline{A}C$	$(A + B)(\overline{A} + C) = (A + B)(B + C)(\overline{A} + C)$
T13	Multiplying	$(A + B)(\overline{A} + C) = AC + \overline{A}B$	$AB + \overline{A}C = (A + C)(\overline{A} + B)$
T14	De Morgan	$\overline{A + B} = \overline{A}\,\overline{B}$	$\overline{AB} = \overline{A} + \overline{B}$

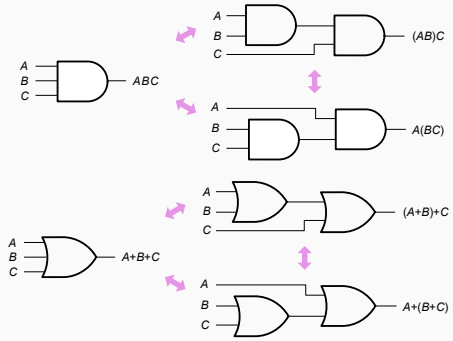
© Muním Zabidi

Commutative Law (T6)



- Exchanging the inputs of an AND or OR does not change the output: $AB = BA$, $A + B = B + A$.

Associative Law (T7)



- Grouping does not matter when the same operation is used throughout.
- Inputs can be reordered or regrouped, which can simplify expressions and circuits.

Proving a Theorem Algebraically

Prove $A + AB = A$

$$
\begin{aligned}
& A + AB \\
&= A \cdot 1 + AB && \text{identity} \\
&= A(1 + B) && \text{distributive} \\
&= A \cdot 1 && 1 + B = 1 \\
&= A && \text{identity}
\end{aligned}
$$

Prove consensus

$$
\begin{aligned}
& AB + \overline{A}C + BC \\
&= AB + \overline{A}C + BC(A + \overline{A}) && \text{complement} \\
&= AB + \overline{A}C + ABC + \overline{A}BC && \text{distributive} \\
&= AB(1 + C) + \overline{A}C(1 + B) && \text{factoring} \\
&= AB + \overline{A}C && \text{identity}
\end{aligned}
$$

Table of Contents

1 Why This Chapter

2 Multivariable Theorems

3 De Morgan's Theorems

4 Expressing Logic Circuits

5 Canonical Forms

6 Don't Cares

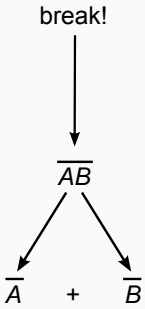
7 Algebraic Simplification

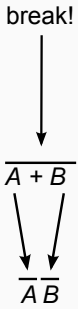
8 Summary

The two theorems

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$
$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

- Break the long bar; the operation under the break changes (AND to OR, OR to AND).



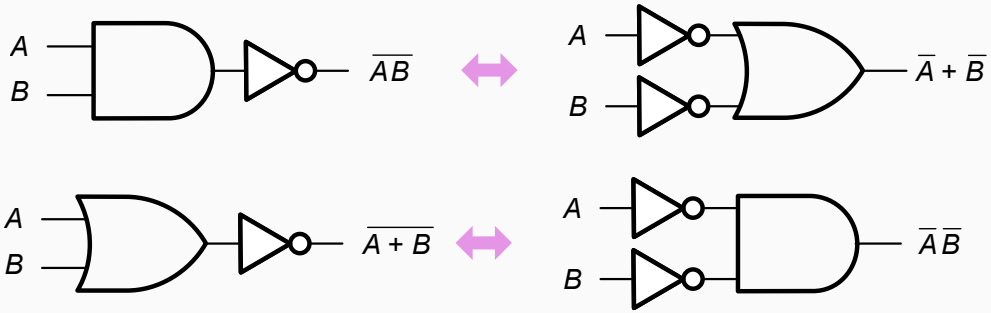


- Valid for any number of variables.

$$\overline{ABC} = \overline{A} + \overline{B} + \overline{C} \qquad \overline{A + BC} = \overline{A}\,(\overline{B} + \overline{C})$$

© Muním Zabidi

De Morgan Changes the Gate



- An AND gate can be redrawn as an OR gate by adding and removing inverters, and vice versa.

Table of Contents

1 Why This Chapter

2 Multivariable Theorems

3 De Morgan's Theorems

4 **Expressing Logic Circuits**

5 Canonical Forms

6 Don't Cares

7 Algebraic Simplification

8 Summary

SOP and POS Forms

Sum-of-products (SOP)

AND terms combined by OR: an AND-OR network.
$F = AB + \overline{A}C$

Product-of-sums (POS)

OR terms combined by AND: an OR-AND network.
$F = (A + B)(\overline{A} + C)$

- POS to SOP: multiply out the terms and simplify.
- SOP to POS: use De Morgan's theorems with involution, $\overline{\overline{A}} = A$.
- SOP is slightly more convenient.

Converting Between SOP and POS

$(A + C)(B + \overline{C})$ to SOP

$$= AB + A\overline{C} + BC + C\overline{C}$$
$$= AB + A\overline{C} + BC$$

$F = A\overline{B} + B\overline{C}$ to POS

$$\overline{F} = (\overline{A} + B)(\overline{B} + C) = \overline{A}\,\overline{B} + \overline{A}C + BC$$

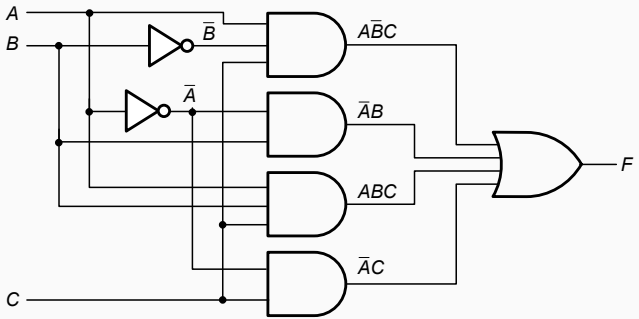
$$F = \overline{\overline{F}} = (A + B)(A + \overline{C})(\overline{B} + \overline{C})$$

Method: complement with De Morgan, expand to SOP, then apply De Morgan again.

Expression to Circuit

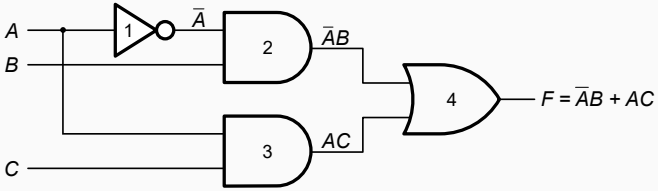
$F = A\overline{B}C + \overline{A}B + ABC + \overline{A}C$

- 3 variables, 4 product terms, 10 literals.
- Each AND needs an AND gate, each OR an OR gate, each complement a NOT.
- Gate inputs: $10 + 4 = 14$ (excluding inverters).



Simplification reduces literals and gate inputs: less power and shorter delay.

Circuit to Expression



1. Name all gate inputs.
2. Write each gate's expression at its output, left to right.
3. The last gate's output is the network equation.

Gate 1: $\overline{A}$. Gate 2: $\overline{A}B$. Gate 3: AC.
Gate 4: $F = \overline{A}B + AC$.

© Muním Zabidi

Truth Table of a Network

A	B	C	$\overline{A}$	$\overline{A}B$	AC	F
0	0	0	1	0	0	0
0	0	1	1	0	0	0
0	1	0	1	1	0	1
0	1	1	1	1	0	1
1	0	0	0	0	0	0
1	0	1	0	0	1	1
1	1	0	0	0	0	0
1	1	1	0	0	1	1

- Columns for every input, every gate output, and the final output; fill the gates in order.

© Munim Zabidi

Table of Contents

1 Why This Chapter

2 Multivariable Theorems

3 De Morgan's Theorems

4 Expressing Logic Circuits

5 **Canonical Forms**

6 Don't Cares

7 Algebraic Simplification

8 Summary

Why Canonical Forms?

- A truth table is a unique description of a function. Expressions and circuits are not.
- Two different-looking expressions may give identical outputs for every input.
- A canonical form writes a function uniquely and systematically from its truth table.

Canonical form	Structure	Built from
SOP	OR of AND terms	minterms
POS	AND of OR terms	maxterms

Minterms and Maxterms

- Minterm: an AND of all variables, each once, true or complemented. It is 1 for exactly one input row.
- Maxterm: an OR of all variables, each once. It is 0 for exactly one input row.
- $A\overline{B}C$ is a minterm; $A\overline{B}$ is not (missing C).

A	B	C	Minterm	Maxterm
0	0	0	$m_0 = \overline{A}\,\overline{B}\,\overline{C}$	$M_0 = A + B + C$
0	0	1	$m_1 = \overline{A}\,\overline{B}C$	$M_1 = A + B + \overline{C}$
0	1	0	$m_2 = \overline{A}B\overline{C}$	$M_2 = A + \overline{B} + C$
0	1	1	$m_3 = \overline{A}BC$	$M_3 = A + \overline{B} + \overline{C}$
1	0	0	$m_4 = A\overline{B}\,\overline{C}$	$M_4 = \overline{A} + B + C$
1	0	1	$m_5 = A\overline{B}C$	$M_5 = \overline{A} + B + \overline{C}$
1	1	0	$m_6 = AB\overline{C}$	$M_6 = \overline{A} + \overline{B} + C$
1	1	1	$m_7 = ABC$	$M_7 = \overline{A} + \overline{B} + \overline{C}$

Canonical SOP from a Truth Table

A	B	C	F	
0	0	0	1	m_0
0	0	1	0	
0	1	0	1	m_2
0	1	1	0	
1	0	0	0	
1	0	1	0	
1	1	0	1	m_6
1	1	1	1	m_7

- Pick the rows where $F = 1$; write each row's minterm; OR them.

$$F = \overline{A}\,\overline{B}\,\overline{C} + \overline{A}B\overline{C} + AB\overline{C} + ABC$$

$$F(A, B, C) = \sum m(0, 2, 6, 7)$$

Expanding to Canonical SOP

- For each missing variable X, multiply the term by $(X + \overline{X}) = 1$ and expand. Drop duplicate terms.
- Canonical forms also let you compare expressions: equal expressions have equal minterm lists.

$F = AB$

$= AB(C + \overline{C}) = ABC + AB\overline{C}$
$= \sum m(6, 7)$

$F = A + B\overline{C}$

$= A(B + \overline{B})(C + \overline{C}) + B\overline{C}(A + \overline{A})$
$= \sum m(2, 4, 5, 6, 7)$

Proving Equivalence with Canonical Forms

Prove $AB + \overline{A}C = (\overline{A} + B)(A + C)$

LHS $= AB(C + \overline{C}) + \overline{A}C(B + \overline{B}) = m_6 + m_7 + m_1 + m_3 = \sum m(1, 3, 6, 7)$

RHS $= \overline{A}C + AB + BC$, expanded: $m_1 + m_3 + m_6 + m_7 = \sum m(1, 3, 6, 7)$

Both give $\sum m(1, 3, 6, 7)$, so the expressions are equal.

Canonical POS

A	B	C	F	
0	0	0	1	
0	0	1	0	M_1
0	1	0	1	
0	1	1	0	M_3
1	0	0	0	M_4
1	0	1	0	M_5
1	1	0	1	
1	1	1	1	

- Pick the rows where $F = 0$; write each row's maxterm; AND them.

$$F = (A + B + \overline{C})(A + \overline{B} + \overline{C})(\overline{A} + B + C)(\overline{A} + B + \overline{C})$$

$$F(A, B, C) = \prod M(1, 3, 4, 5)$$

- To add a missing variable C to a sum term: $A + B = (A + B + C)(A + B + \overline{C})$.

Converting Between Canonical Forms

- A function is 1 at its minterm indices and 0 at its maxterm indices.
- The two index lists are complementary: swap to the indices that are missing.

SOP to POS

$F = \sum m(3, 4, 5, 6, 7)$. Missing indices: 0, 1, 2.
$F = \prod M(0, 1, 2)$

POS to SOP

$F = \prod M(1, 2, 5)$. Missing indices: 0, 3, 4, 6, 7.
$F = \sum m(0, 3, 4, 6, 7)$

Table of Contents

1 Why This Chapter

2 Multivariable Theorems

3 De Morgan's Theorems

4 Expressing Logic Circuits

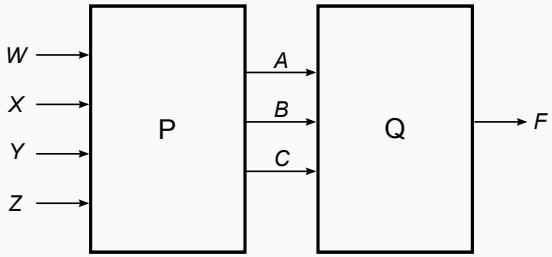
5 Canonical Forms

6 **Don't Cares**

7 Algebraic Simplification

8 Summary

Incompletely Specified Functions



A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	$\times$
1	1	1	$\times$

$$F = \sum m(0,1,5) + \sum d(6,7)$$

$$F = \prod M(2,3,4) \prod D(6,7)$$

- Block P drives A, B, C into block Q; P never produces $ABC = 110$ or 111, so those inputs of Q cannot arise.
- Such inputs, or ones whose output does not matter, are don't cares, written $\times$, $-$ or . They are don't cares because the input cannot occur, not because the output is unimportant.
- Assign them 0 or 1 freely to simplify the function and cut gates.

Table of Contents

1 Why This Chapter

2 Multivariable Theorems

3 De Morgan's Theorems

4 Expressing Logic Circuits

5 Canonical Forms

6 Don't Cares

7 **Algebraic Simplification**

8 Summary

Simplification Procedure

Step	Action	Before	After
1	Convert to SOP	$(A+B)(C+D)$	$AC+AD+BC+BD$
2	Drop exact duplicates	$A\overline{B}+CB+A\overline{B}$	$A\overline{B}+CB$
3	Drop null terms	$A(\overline{A}+B)$	$A\overline{A}+AB=AB$
4	Absorb contained terms	$AB+ABC$	$AB(1+C)=AB$
5	Combine terms differing in one literal	$ABC+AB\overline{C}$	$AB(C+\overline{C})=AB$

© Muním Zabidi

Simplification Examples

$F = (A + B)(A + \overline{B})(\overline{A} + C)$

$= [A + A(\overline{B} + B) + 0](\overline{A} + C)$
$= A(\overline{A} + C) = A\overline{A} + AC$
$= AC$

$F = \overline{A}B + A\overline{B}C + BC$

$= \overline{A}B + (A\overline{B} + B)C = \overline{A}B + (A + B)C$
$= \overline{A}B + AC + BC = \overline{A}B + AC$

$F = AB + \overline{A}C + BC$

$= AB + \overline{A}C + BC(A + \overline{A})$
$= A(B + BC) + \overline{A}(C + BC)$
$= AB + \overline{A}C$
This proves the consensus theorem.

$F = \overline{(A + \overline{B})(\overline{C} + D)}$ to SOP

$= \overline{(A + \overline{B})} + \overline{(\overline{C} + D)}$
$= \overline{A}B + C\overline{D}$

Back to the Opening Question

What makes one circuit “better”?

- Fewer literals and gate inputs: less area and cost.
- Fewer gates in the path: shorter delay.
- Less switching hardware: lower power.

Boolean algebra gets us from the first, correct-but-large expression to a smaller one with the same truth table.

Table of Contents

1 Why This Chapter

2 Multivariable Theorems

3 De Morgan's Theorems

4 Expressing Logic Circuits

5 Canonical Forms

6 Don't Cares

7 Algebraic Simplification

8 **Summary**

Summary

- A function can be a truth table, an expression or a circuit: all carry the same information.
- Canonical SOP is a sum of minterms; canonical POS is a product of maxterms, dual to SOP.
- De Morgan's theorems link AND with OR under complementation.
- Don't cares let you simplify further.
- Algebraic simplification is systematic but error-prone by hand.

Looking ahead

Chapter 6 introduces Karnaugh maps, a visual tool that automates minimisation.